



CS-5450
MOBILE PROGRAMMING
CHALLENGE 3 - React Native / Expo Community
Portal
GROUP -9

Project Description

CommunityHub App is a mobile community portal application developed using React Native and Expo. The main purpose of this application is to provide a digital platform where users can register, log in, communicate through discussion forums, explore events, browse member profiles, and join community interest groups.

The application is designed as a mini portal that supports shared interests, discussion-based interaction, member networking, and event participation. Firebase Authentication and Cloud Firestore are used to store and manage user accounts, forum posts, replies, events, groups, and member information.

This project was developed for Group 9 as part of Challenge 3 in CS5450 Mobile Programming at Lakehead University.

Technologies Used

- React Native
- Expo
- JavaScript
- Firebase Authentication
- Cloud Firestore
- React Navigation
- AsyncStorage
- Android Studio
- Android Emulator

Main Features

- User registration and login using Firebase Authentication.
- Personalized dashboard for signed-in users with community statistics and recent activity.
- Discussion forums with post creation, replies, likes, and real-time updates.
- Event section with event creation, join/leave functionality, category filters, and search.
- Member directory with search by name or interests and profile modal view.
- Groups section with join/leave functionality and live member count.
- Dark-themed responsive interface for mobile and web through Expo.

- Firestore security rules for signed-in access and admin-controlled actions.

Firebase Usage

Firebase is used in this project for both authentication and database services. Firebase Authentication allows users to create accounts and sign in using email and password.

Cloud Firestore is used to store collections such as users, forums, events, groups, and replies. The project uses Firestore real-time listeners through `onSnapshot`, so data changes in forums, events, and groups are reflected immediately in the application UI.

The Firebase configuration is stored in `src/services/firebase.js`, where the Firebase app is initialized and the authentication and database instances are exported for use throughout the project.

How to Configure and Run the Project

1. Open the project folder in VS Code or Android Studio.
2. Open a terminal inside the `community-hub` folder.
3. Run the following command to install dependencies:

```
```bash
```

```
npm install
```

```
```
```

4. Start the Expo development server:

```
```bash  

npx expo start

```
```

5. To run the project on Android, press:

```
```bash  

a

```
```

6. To run the project on web, press:

```
```bash  

w

```
```

7. You can also scan the QR code using Expo Go on a physical device after starting the Expo server.

6. Project Structure

The project follows a structured React Native folder organization. Authentication, screens, Firebase services, and utility functions are separated to keep the code clean and easy to manage.

```
```text
```

```
community-hub/
```

```
|— App.js
```

```
|— index.js
```

```
|— app.json
```

```
|— package.json
```

```
|— package-lock.json
```

```
|— firestore.rules
```

```
|— README.md
```

```
└─ src/
```

```
 |— hooks/
```

```
 └─ useAuth.js
```

```
 |— services/
```

```
 |— firebase.js
```

```
 |— authService.js
```

```
 └─ firestoreService.js
```

```
 |— screens/
```

```
 |— AuthScreen.js
```

```
 |— DashboardScreen.js
```

```
| |— ForumsScreen.js
| |— EventsScreen.js
| |— MembersScreen.js
| |— utils/
| |— helpers.js
```

...

## **7. GitHub Repository**

The full project is uploaded to GitHub.

GitHub Link:

<https://github.com/mvahora217/community-hub-cs5450.git>

## **8. Firebase Screenshots**

Firebase screenshots will be added later to show the backend collections and authentication setup used in the project.

Suggested screenshots:

- Firebase Authentication users
- Firestore `users` collection
- Firestore `forums` collection
- Firestore `events` collection
- Firestore `groups` collection

## **9. Code Explanation**

### **9.1. App.js and Navigation Setup**

The `App.js` file is the entry point of the application. It sets up the `NavigationContainer`, bottom tab navigation, stack navigation for forum threads, and the authentication gate that decides whether the user sees the login screen or the main app screens.

The app includes the following main navigation tabs:

- Dashboard
- Forums
- Members
- Events
- Groups

### **9.2. Firebase Initialization**

The `firebase.js` file initializes the Firebase application and configures authentication persistence using AsyncStorage. It also exports the `auth` and `db` instances so the rest of the app can communicate with Firebase Authentication and Cloud Firestore.

### **9.3. Authentication Service**

The `authService.js` file manages user registration, login, logout, authentication state changes, and profile retrieval. When a user registers, the project also creates a Firestore profile document containing user details such as name, email, avatar initials, role, biography, interests, and statistics.

## 9.4. Firestore Service

The `firestoreService.js` file contains the main database operations for the project. It manages:

- user profile retrieval and updates,
- forum post creation and likes,
- reply creation for threads,
- event creation and join/leave actions,
- group join/leave actions,
- seeding default groups,
- and real-time subscriptions using Firestore listeners.

## 9.5. Screen Implementation

The project includes separate screen files for each major feature:

- `AuthScreen.js` handles sign in and registration forms.
- `DashboardScreen.js` displays a personalized home screen with user stats, recent discussions, upcoming events, and joined groups.
- `ForumsScreen.js` manages forum discussions, replies, likes, and post categories.
- `EventsScreen.js` displays events, search, filters, and event creation functionality.
- `MembersScreen.js` provides the member directory and also includes group-related functionality.



## **11. Security Rules**

The project includes Firestore security rules to help protect user data and control access. The rules require users to be signed in before reading or writing most data, and they allow certain actions such as deleting groups or managing privileged records only for admins.

This improves the security of the portal and supports the assignment requirement for security integration.

## **12. Testing**

The application was tested using Expo and Android/web-compatible environments. The project is configured to run on Android, iOS, and web platforms through Expo, and the interface is designed using a consistent dark theme.

Core features tested in the project include:

- user registration and login,
- dashboard loading,
- real-time forum updates,
- event joining and leaving,
- member search,
- and group participation.

### **13. Assignment Requirements Coverage**

This project satisfies the main Challenge 3 requirements in the following ways:

- User Registration & Login: Implemented with Firebase Authentication.
- Personalized Dashboard: Implemented in the dashboard screen using user profile data and Firestore content.
- Search Functionality: Implemented in member, group, and event-related screens.
- Responsive Design: Built using React Native and Expo with support for Android, iOS, and web.
- Security Integration: Implemented using Firebase Authentication and Firestore security rules.
- Cloud Data Storage: Implemented using Cloud Firestore.

### **14. Submission Package**

The final submission package contains:

- Full React Native / Expo project files
- Source code for screens, services, hooks, and utilities
- Firebase configuration file
- Firestore security rules
- README file
- GitHub repository link [user query]

## **15. Future Improvements**

Possible future improvements for this project include:

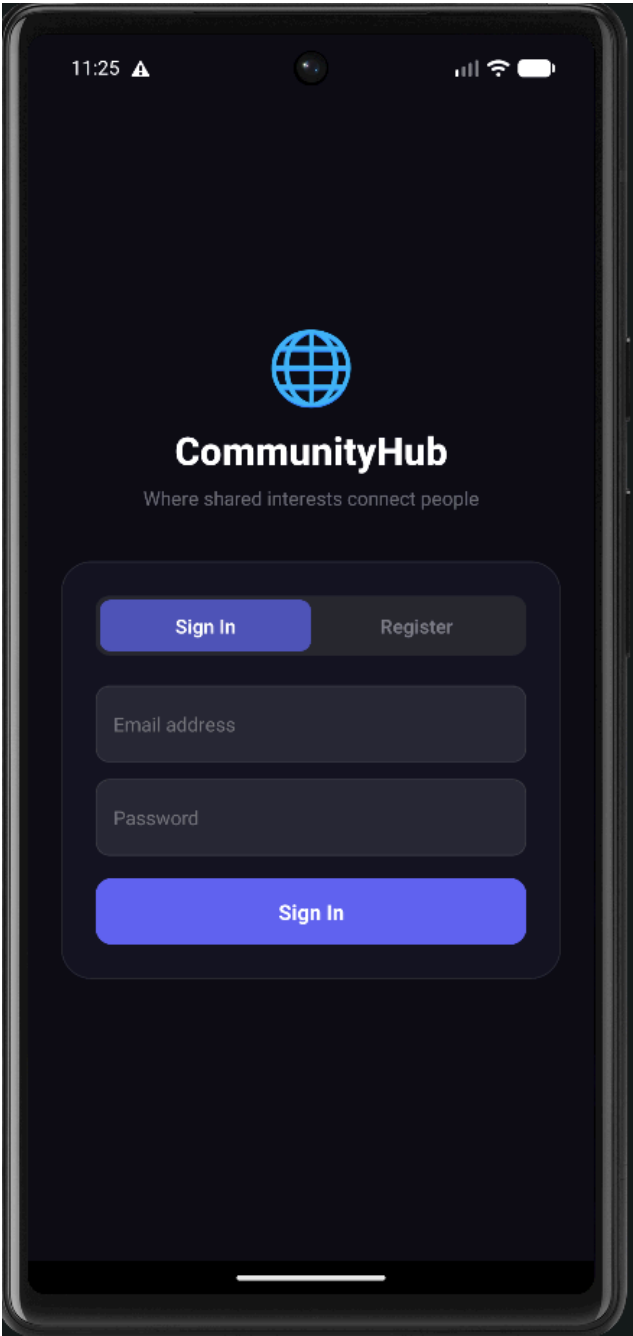
- profile photo upload using Firebase Storage,
- push notifications for replies and event reminders,
- direct messaging between community members,
- admin dashboard for moderation and management,
- pagination for large lists of users and posts,
- and export/reporting features for portal data.

## **16. Conclusion**

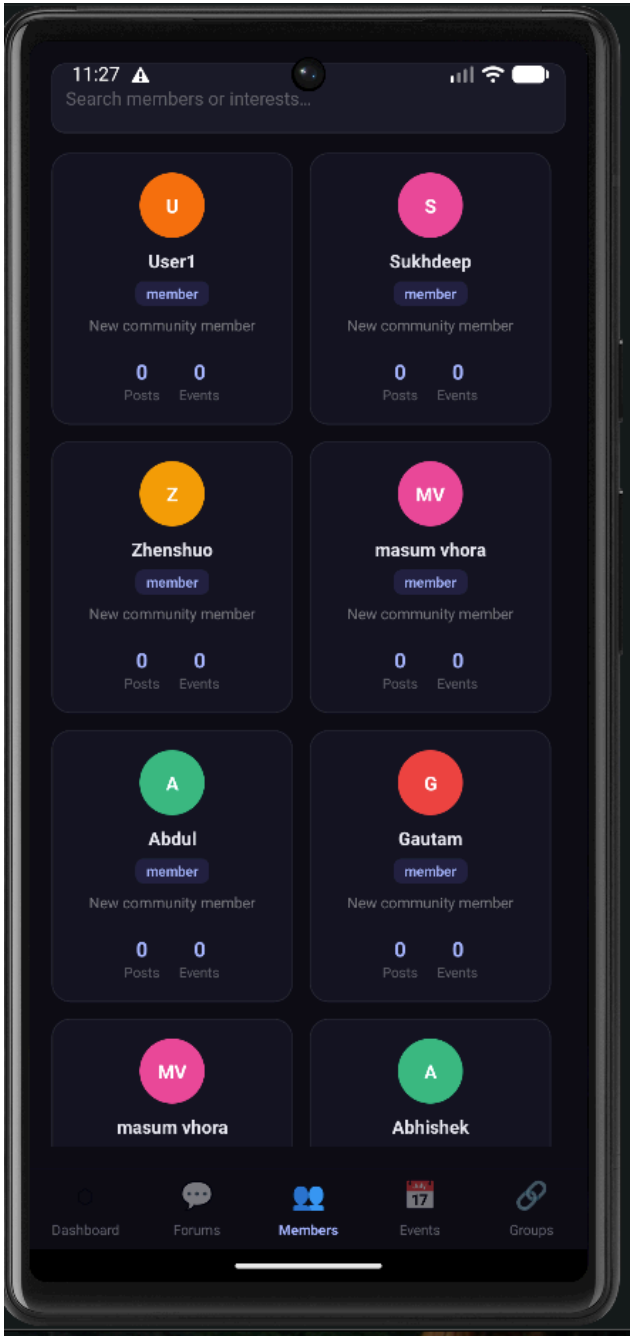
This project meets the requirements of Challenge 3 by creating a mini community portal using React Native, Expo, Firebase Authentication, and Cloud Firestore. The application provides secure login, a personalized dashboard, forums, events, member search, group interaction, and real-time data support in a structured mobile application.

Screenshots

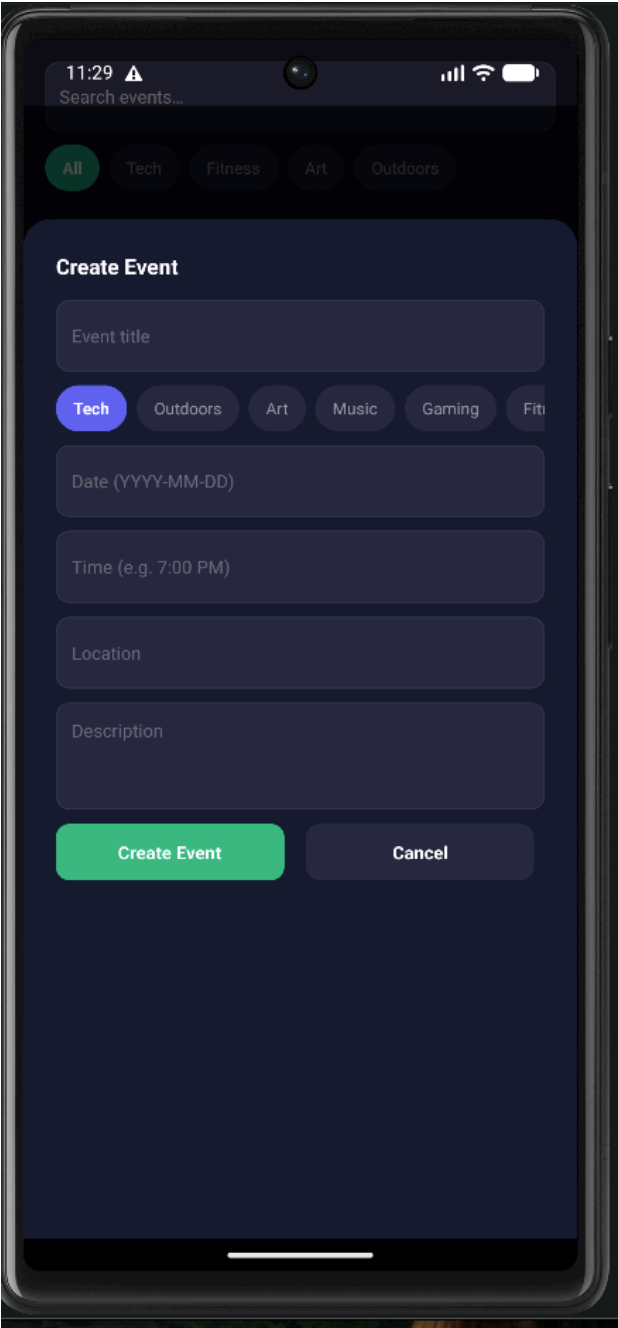
Login/Register



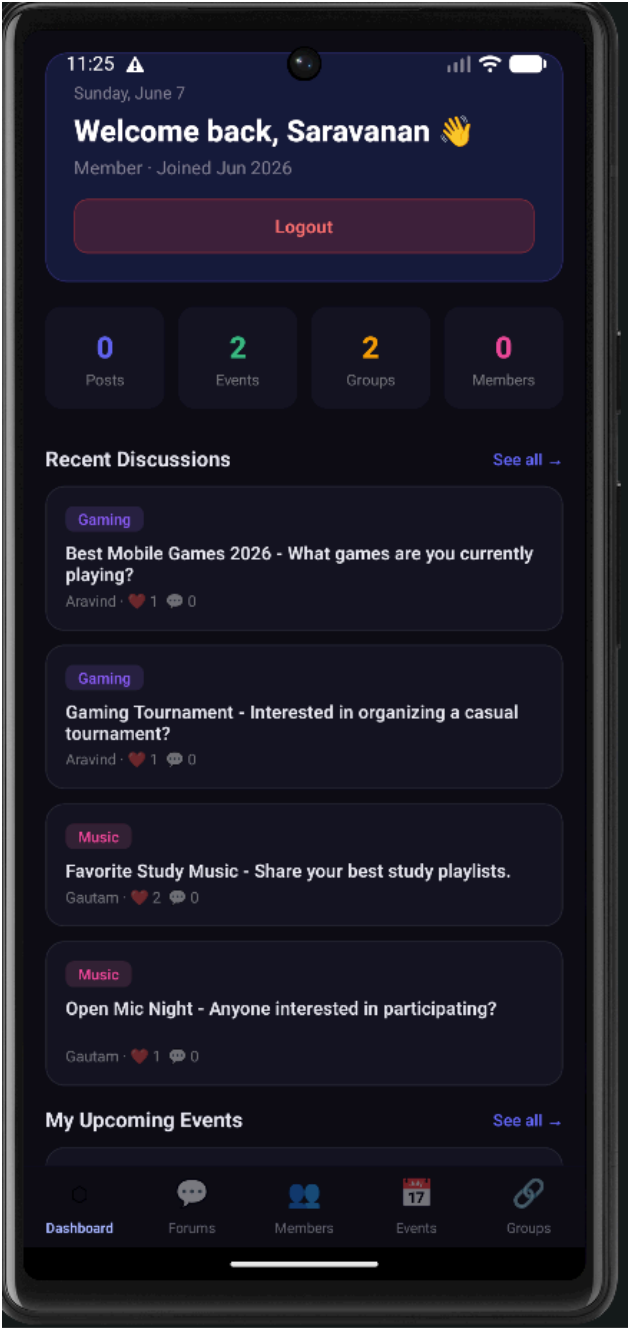
Members Screen



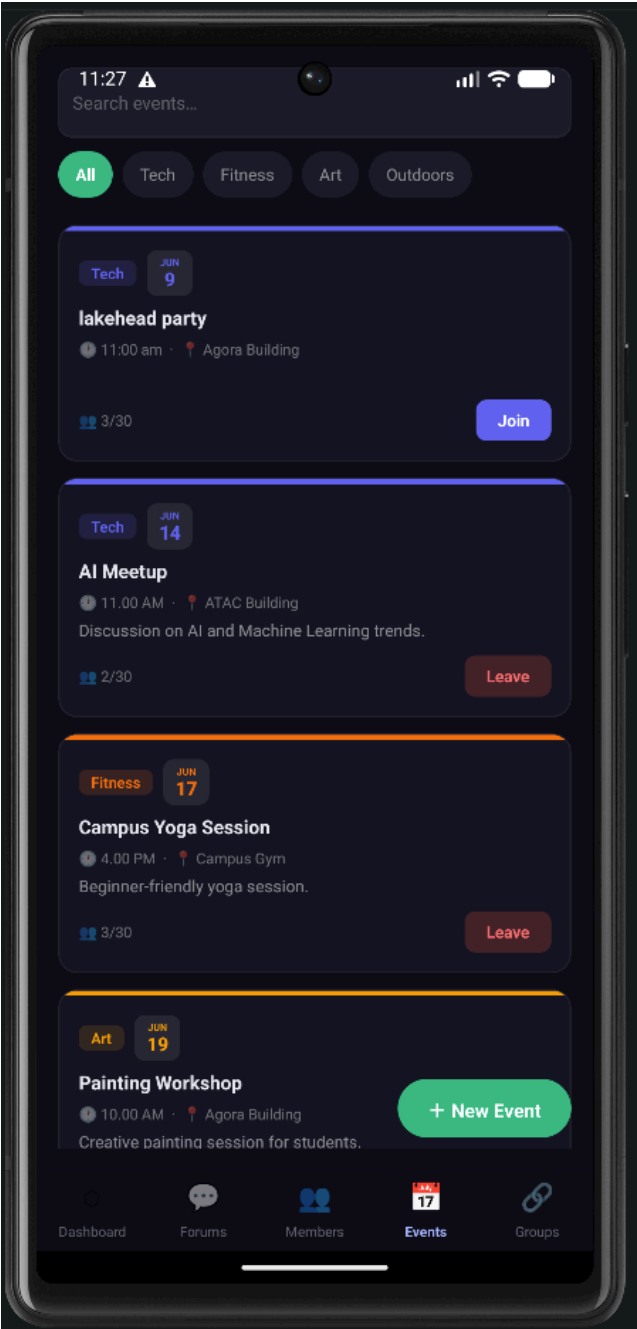
## Create Events Screen



## Dashboard.



Events Screen



Forums

